

Hashing

Search key $\rightarrow h(k) \rightarrow$

Contents corresponding
to the search
key can be
found

Static hashing hashing

There are m blocks/buckets

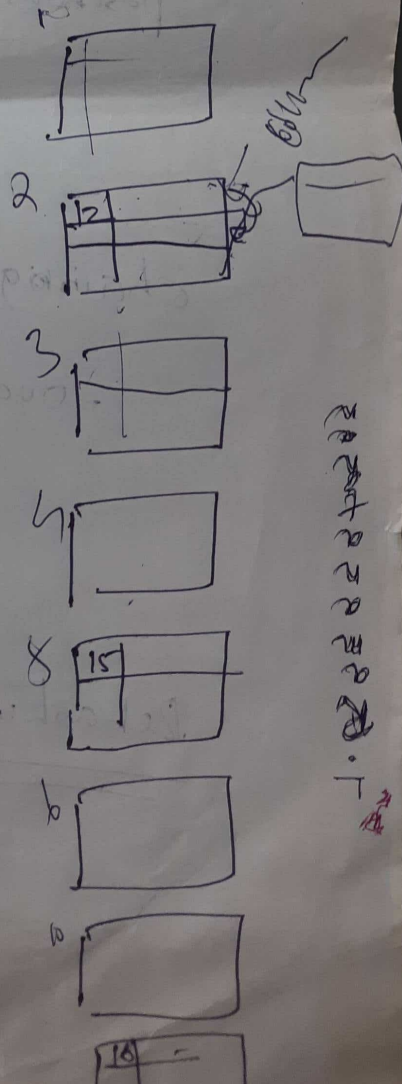
$$h(k) = k \bmod m$$

$$k = 12$$

$$h(k) = 12 \bmod 10 = 2$$

$$k = 15 \quad h(k) = 15 \bmod 10 = 5$$

$$k = 18 \quad h(k) = 18 \bmod 10 = 8$$



Collision

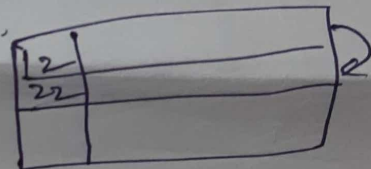
$$K = 22$$

$$22 \bmod 10 = 2$$

Collision 6 Curs

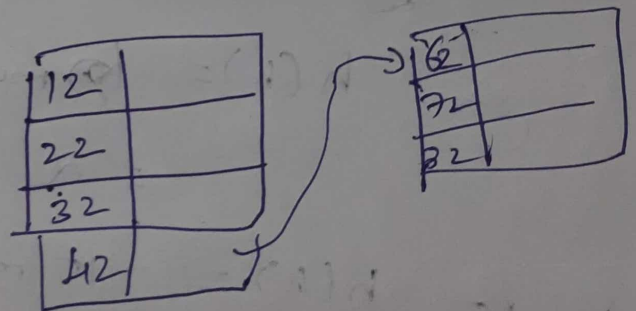
Collision Resolution

open addressing - check subsequent positions until an empty position is found



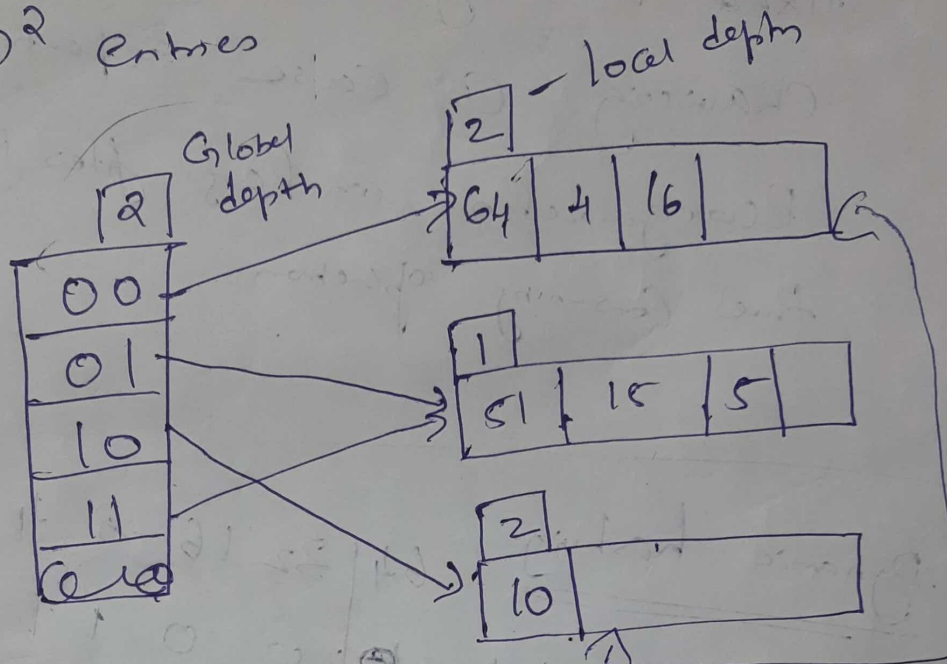
chaining

overflow blocks



Rehashing

Consider the last two bits
and build an access structure having
 2^2 entries



Insert 12

12 hashes to 1100 → bucket 1

Insert 18

18 hashes to 10010 → goes to bucket 3

Insert 20

16 8 4 2 1
1 0 1 0 0

↓
hashes to bucket 1

overflow occurs

Disadvantages of static hashing

Fixed number of blocks

Chaining - On Cause Sequential search

Reorganization on large files is a

time consuming operation

Dynamic hashing

Extendable hashing

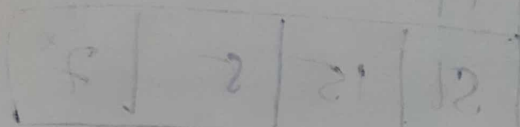
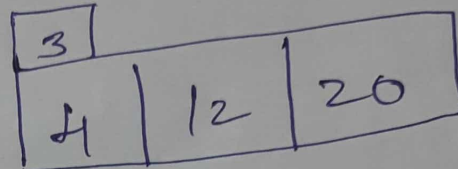
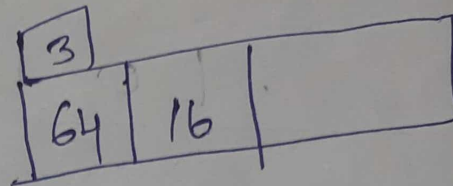
$$h(k) =$$

$$k=12 \quad h(12) = 001100$$

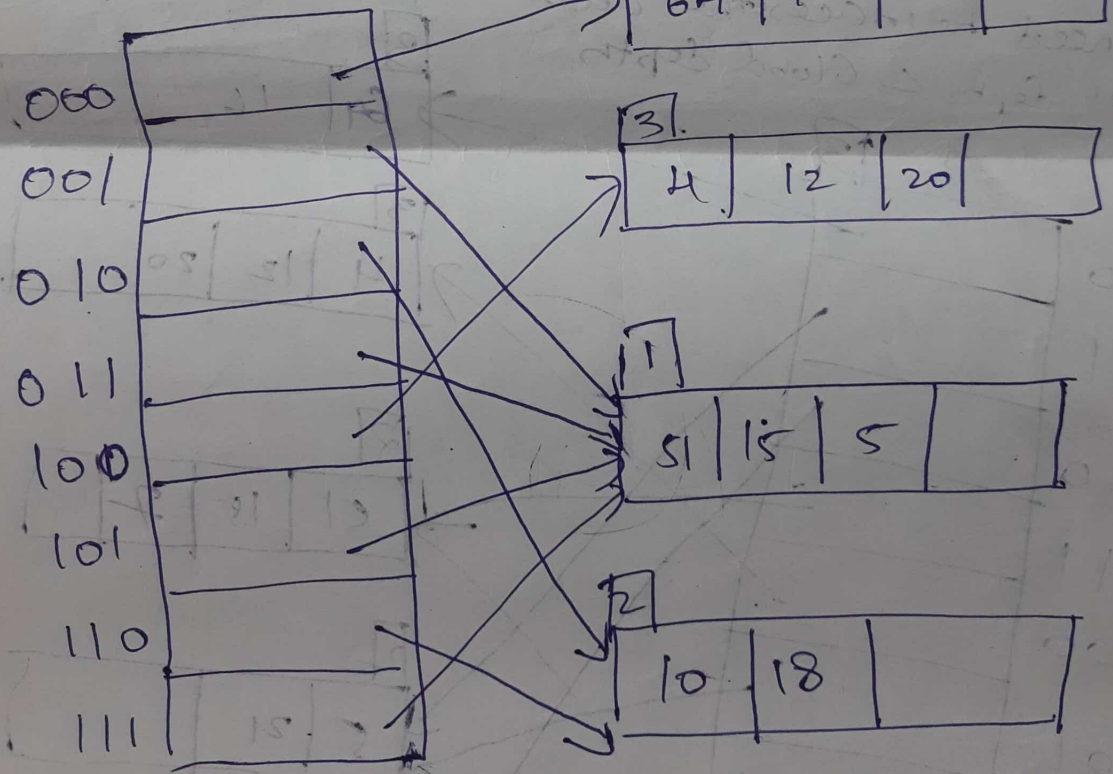
64	32	16	8	4	2	1
0	0	0	1	1	0	0

	64	32	16	8	4	2	1
1 ← 4	0	0	0	0	1	0	0
5	0	0	0	0	1	0	1
10	0	0	0	1	0	1	0
x 12	0	0	0	1	1	0	0
15	0	0	0	1	1	1	1
1 ← 16	0	0	1	0	0	0	0
x 18	0	0	1	0	0	1	0
51	0	1	1	0	0	1	1
1 ← 64	1	0	0	0	0	0	0

1) split ~~the~~ bucket 1 and increase the local depth by 1



Other buckets have the same local depth.
Increase Global depth to 3 $\rightarrow 2^3$

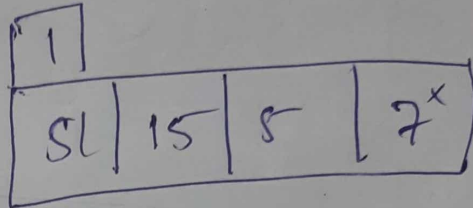


The access structure doubles and
Number of pointer also doubles.

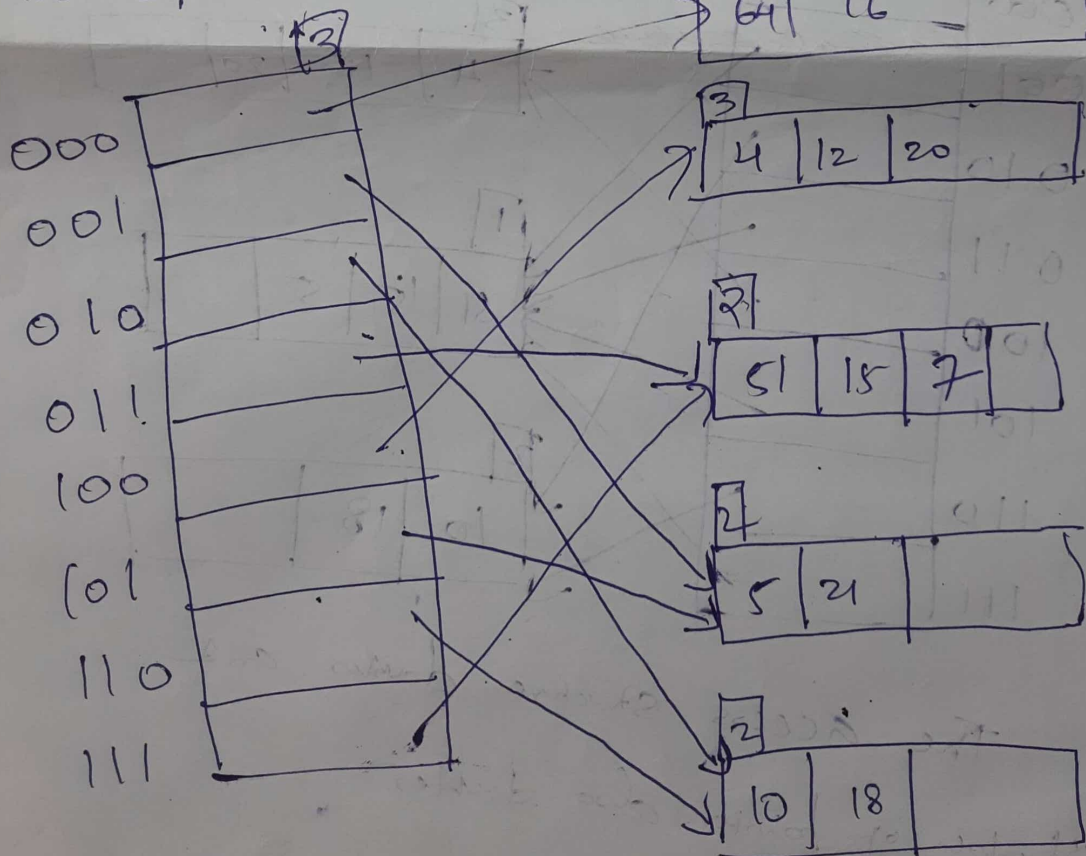
Insert 71 followed by 21

7 0111

21 10101



Inserting 21 forces the bin to split
 No need to increase the access structure
 local depth < Global depth



Insert 28 followed by 44.

8, 5, 1, 7, 3, 12, 9, 6

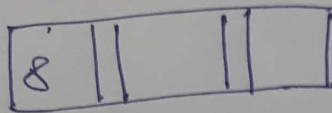
Insert

Start with
leaf node

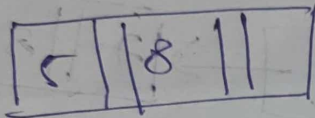
$p=3$ $k=2$

$P_{leaf} = 2$

Insert 8



Insert 5



Insert 1 $\Rightarrow$

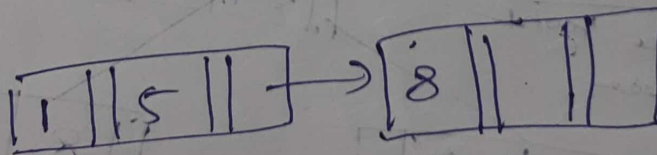
overflow happens.

$$\text{split} = \left\lceil \frac{(p_{leaf} + 1)}{2} \right\rceil$$

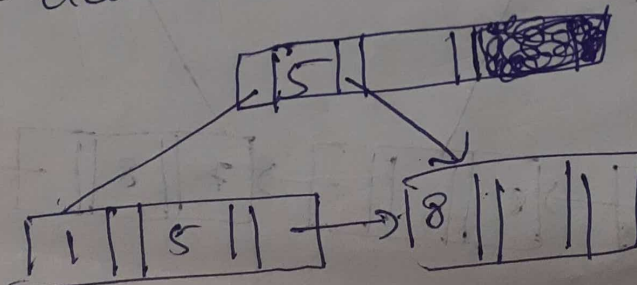
$$\frac{2+1}{2} = \left\lceil \frac{3}{2} \right\rceil = 1.5 = 2$$

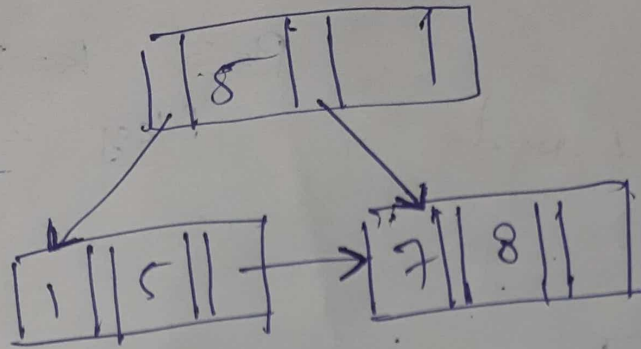
Minimum two values in the original node
remaining in the ~~split~~ ^{split} leaf node

split



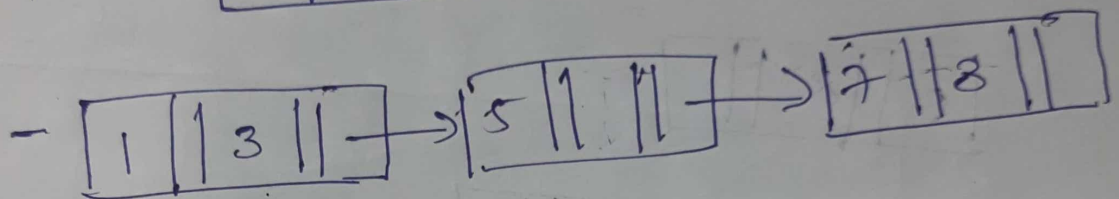
Insert ~~8~~ Create the root node



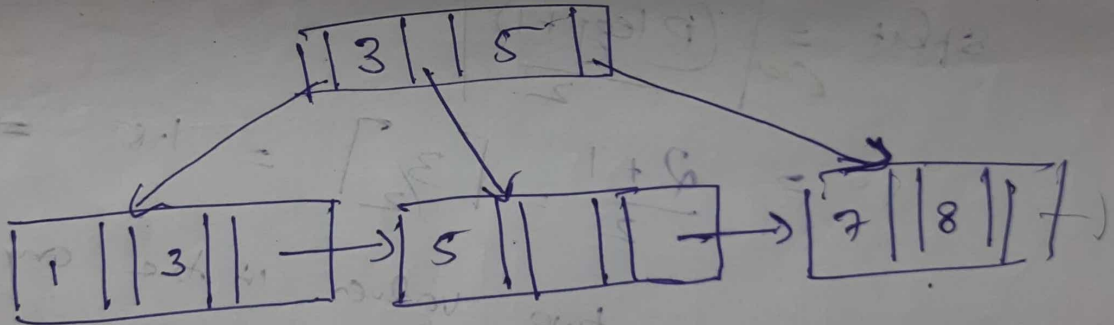


Insert 3

split - [1 | 5 |]

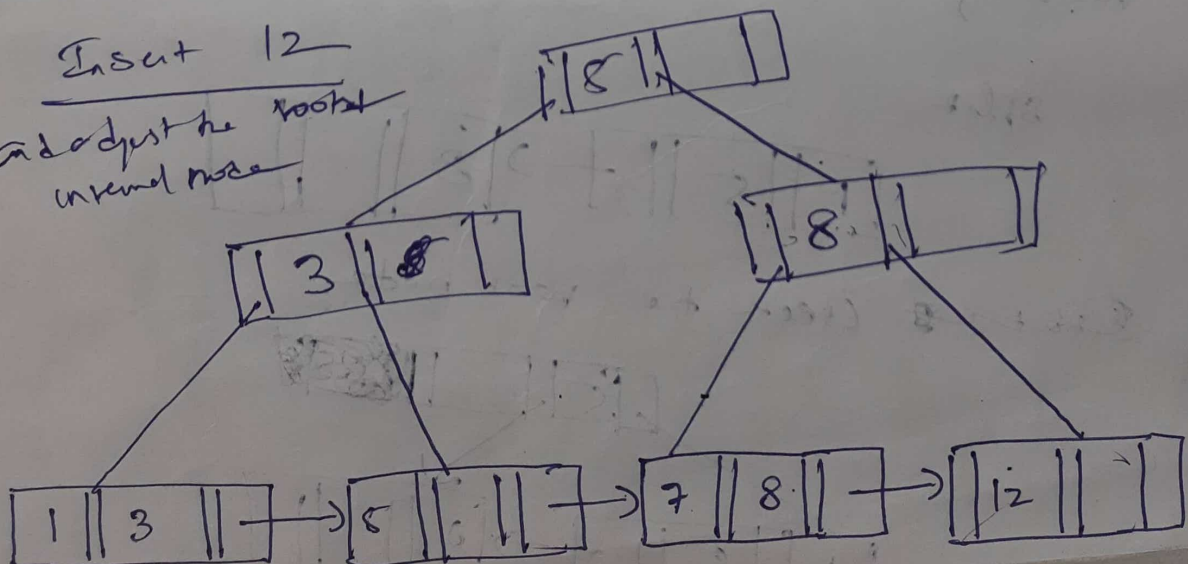


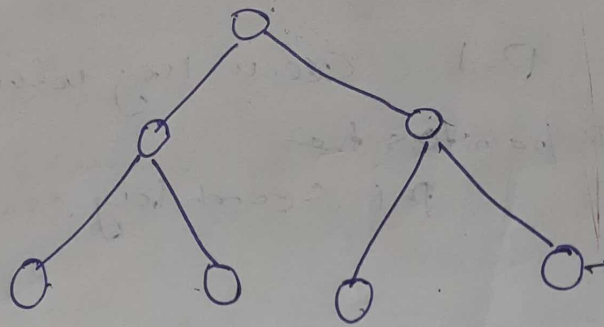
Root node should be adjusted for all three nodes



Insert 12

and adjust the root and internal nodes





self-balancing
efficient search

$$n = 7$$

height $\leq \log n$ then tree is balanced



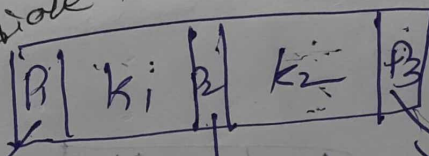
Node - Is size of a block

If indexing is according to primary key

k records +

k+1 pointers in a node

Stores
root node &
Intermediate node



points
equal to k_1

greater than k_1
and less than equal to k_2

X is greater than k_2

1) Always

$$k_1 < k_2 < \dots < k_n$$

Sol.

2) P_1 points values $X \leq k_1$

3) P_2 points values $k_1 < X \leq k_2$

4) P_3 points to value $k_2 < X$

5) Each internal node has atleast

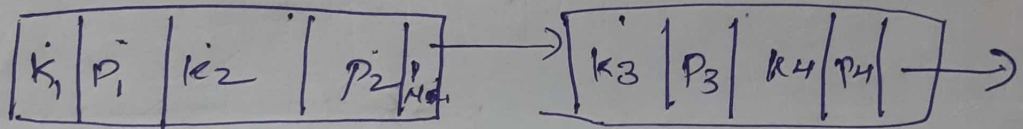
$$\lceil \frac{P}{2} \rceil \text{ pointers}$$

$$\lceil \frac{3}{2} \rceil =$$

6) Ancest P-1 Given key values
 with P pointers then
 P-1 Search key

Leaf nodes

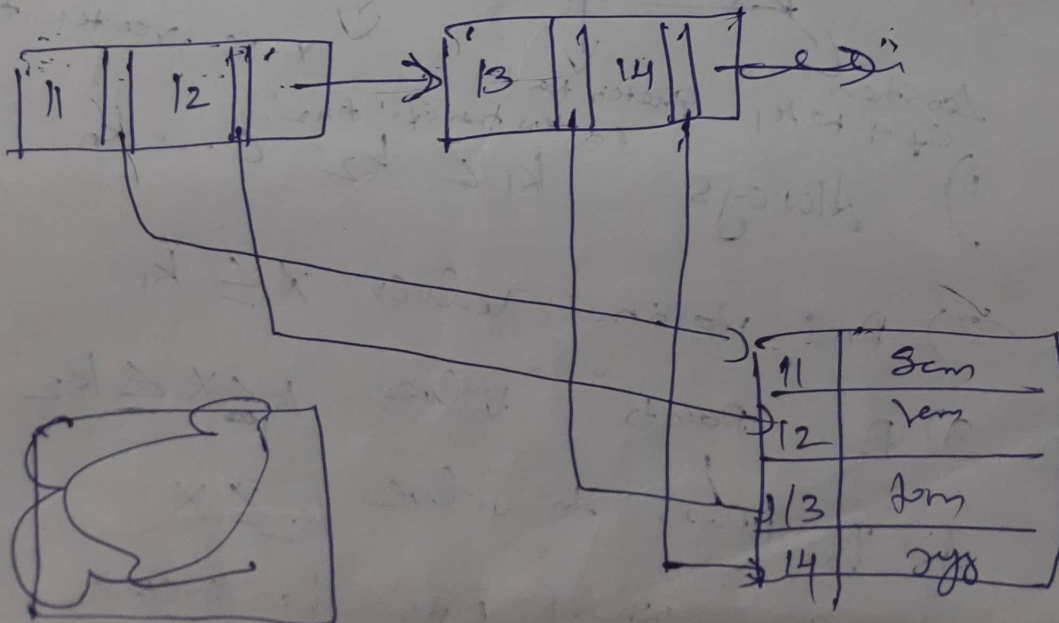
5 touch



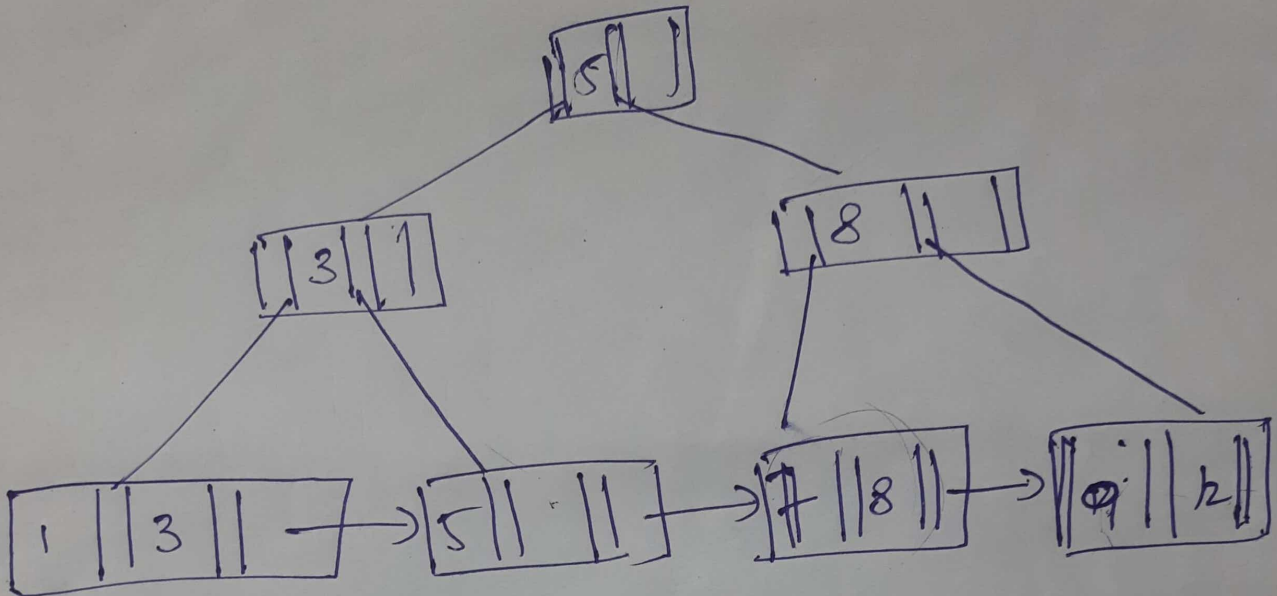
Even leaf node

$$k_1 < k_2 < k_3 < \dots < k_{q-1}$$

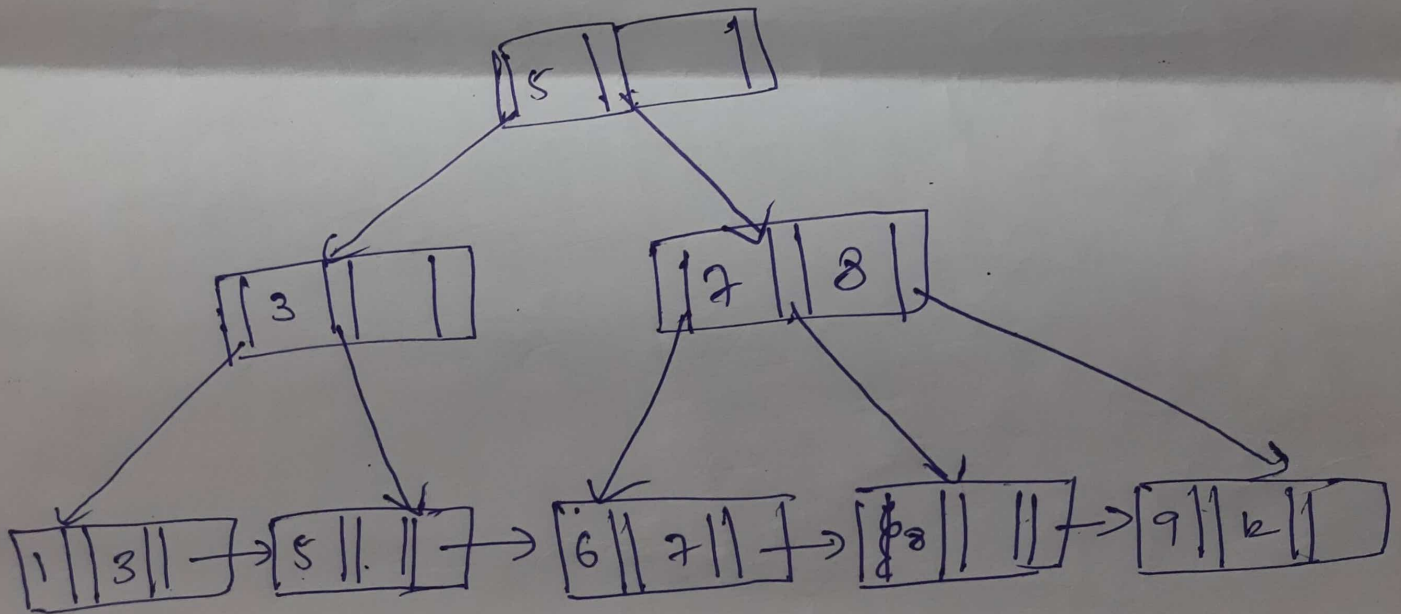
Even & leaf P values



Insert - 6

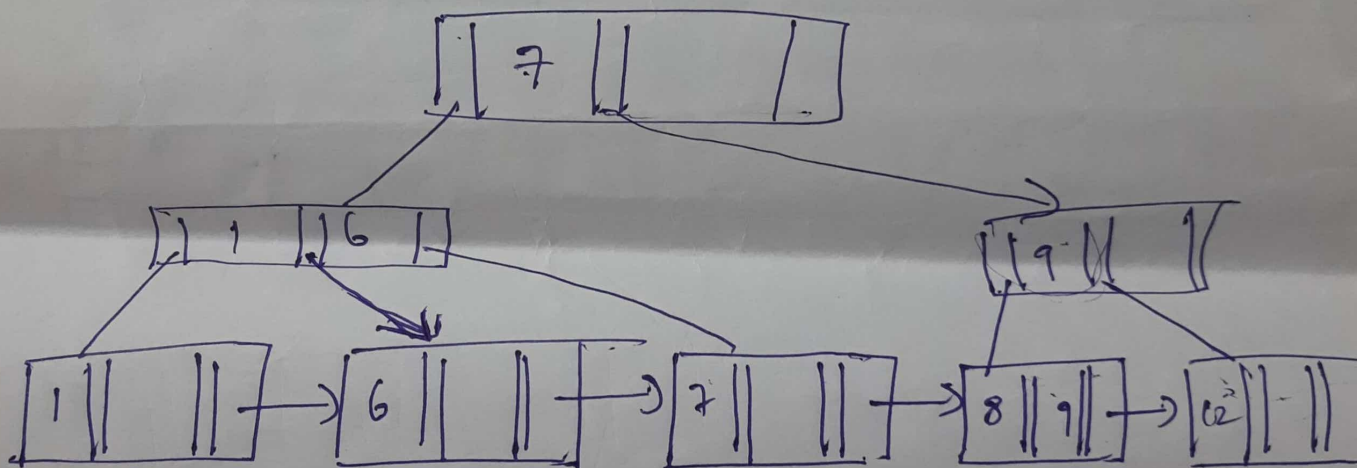
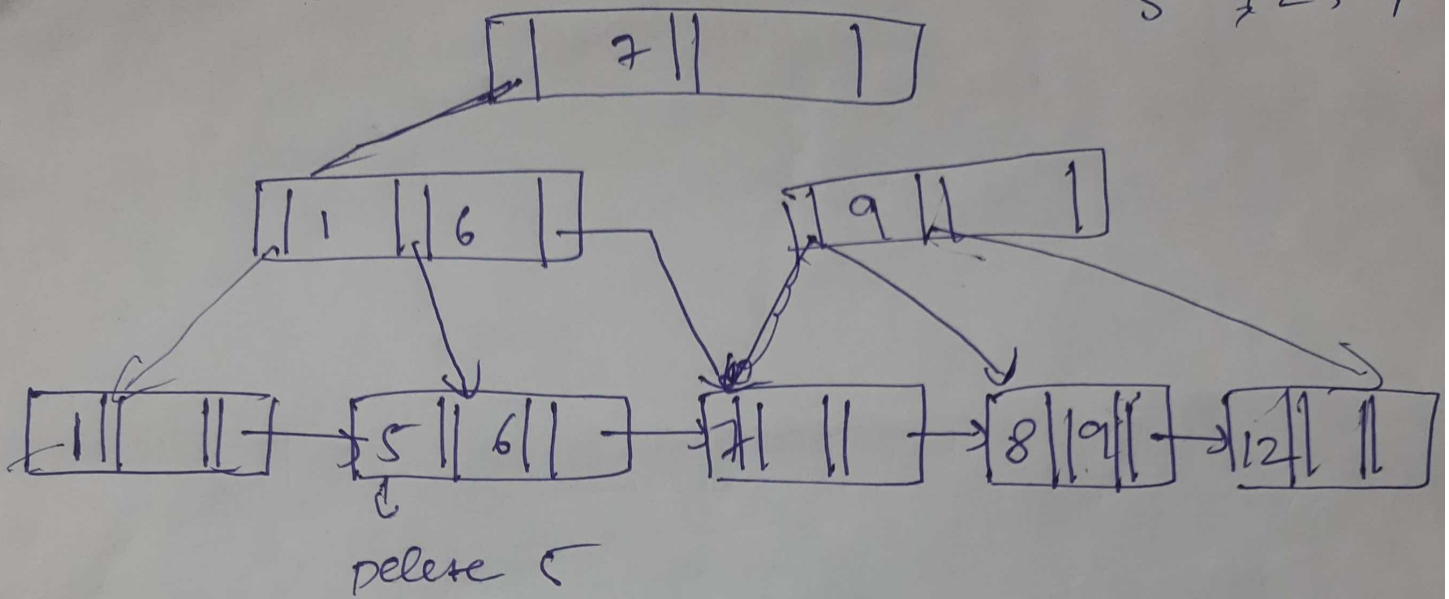


Insert 6

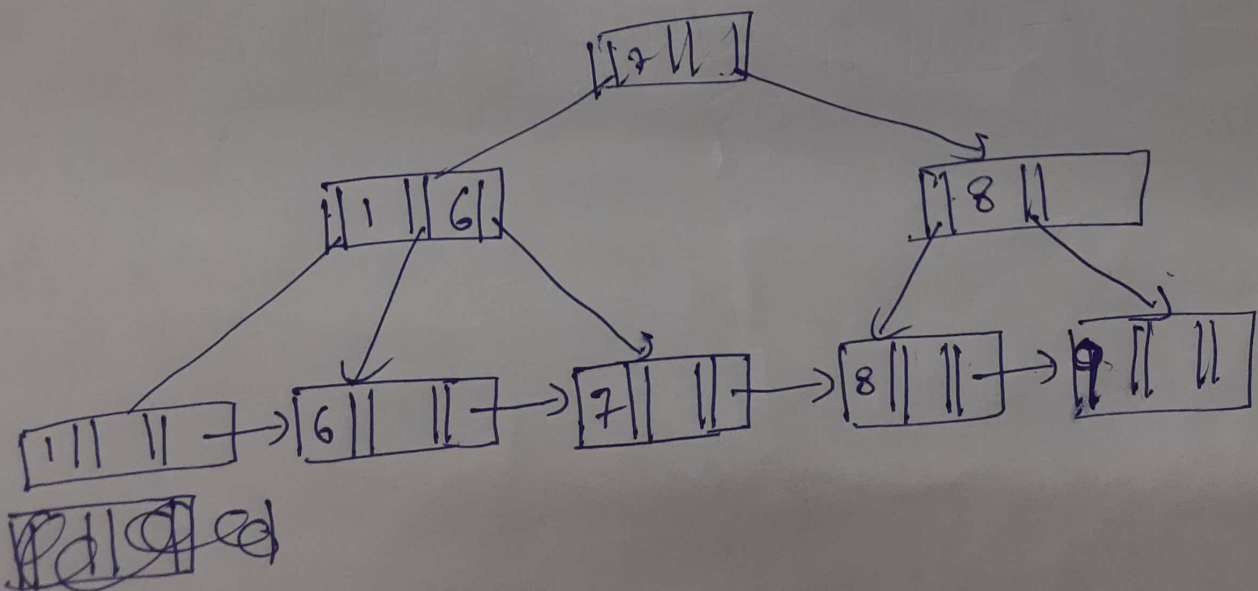


Deletion in a B+ tree

5, 12, 9



delete 12 - underflow - redistribute



Delete 9

